

SKILL-4

Roll No. : 2520030437

Name : Bandla Venkata Rama Vinay

SEC-8A

SKILL-4:

Aim: Develop C programs to process a command entered by the user by performing command tokenization and generating a simple parse-tree representation of the command structure.

The first program separates a command into individual tokens using **strtok()**. The second program identifies the first token as the command and represents the remaining tokens as arguments in a parse-tree format, followed by syntax validation.

Procedure :

1. Create the command-tokenization program in **Skill-4.c**.
2. Read a command using **fgets()** and remove its trailing newline.
3. Use **strtok()** with space and tab delimiters to display the individual tokens.
4. Create the second program in **Skill-4-1.c** to build a command and argument structure.
5. Identify the first token as the command and the following tokens as arguments.
6. Display the parse tree and report whether syntax validation is successful.

Output - Command Tokenization :

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ ./Skill_4
Enter a command: history

Tokens:
Token: [history]
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ ./Skill_4
Enter a command: ls -l /home

Tokens:
Token: [ls]
Token: [-l]
Token: [/home]
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ |
```

The first execution demonstrates tokenization of two commands. The command **history** produces one token, while **ls -l /home** produces three tokens: **ls**, **-l**, and **/home**.

Code - Command Tokenization:

```
Skill > c Skill-4.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char input[200];
7      char *token;
8
9      printf("Enter a command: ");
10     fgets(input, sizeof(input), stdin);
11
12     input[strcspn(input, "\n")] = '\0';
13
14     if (strlen(input) == 0)
15     {
16         printf("Empty input.\n");
17         return 0;
18     }
19
20     printf("\nTokens:\n");
21
22     token = strtok(input, " \t");
23
24     while (token != NULL)
25     {
26         printf("Token: [%s]\n", token);
27         token = strtok(NULL, " \t");
28     }
29
30     return 0;
31 }
```

Output - Parse Tree Generation :

```
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ ./Skill_4-1
Enter command: ls -l /home

Parse Tree:
COMMAND: ls
|
+--- ARGUMENT: -l
|
+--- ARGUMENT: /home

Syntax validation successful.
Execution structure generated.
vinay@DESKTOP-0CVM4SE:/mnt/c/Testing/OSSP_Testing/skill$ |
```

The second execution processes **ls -l /home**. The parse tree identifies **ls** as the command and **-l** and **/home** as arguments. The program then reports **Syntax validation successful.** and **Execution structure generated.**

Code - Parse Tree and Syntax Validation:

Skill > c Skill-4-1.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char input[200];
7
8      printf("Enter command: ");
9      fgets(input, sizeof(input), stdin);
10
11     input[strcspn(input, "\n")] = '\0';
12
13     if (strlen(input) == 0)
14     {
15         printf("Syntax Error: Empty command.\n");
16         return 0;
17     }
18
19     printf("\nParse Tree:\n");
20
21     char *command = strtok(input, " ");
22
23     if (command == NULL)
24     {
25         printf("Syntax Error.\n");
26         return 0;
27     }
28
29     printf("COMMAND: %s\n", command);
30
31     char *argument = strtok(NULL, " ");
32
33     while (argument != NULL)
34     {
35         printf("    |\n");
36         printf("    +--- ARGUMENT: %s\n", argument);
37         argument = strtok(NULL, " ");
38     }
39
40     printf("\nSyntax validation successful.\n");
41     printf("Execution structure generated.\n");
42
43     return 0;
44 }
```

Functions and Program Operations

Function / Operation	Purpose
fgets()	Reads the command entered by the user.
strcspn()	Locates the newline character so it can be removed.
strlen()	Checks whether the entered command is empty.
strtok()	Splits the command into tokens using the selected delimiters.
Parse-tree output	Represents the command followed by its arguments hierarchically.
Syntax validation	Reports whether the command structure was processed successfully.

Observation :

- The tokenization program accepts a command and displays each token separately.
- **history** is treated as a single token.
- **ls -l /home** is separated into the command **ls** and the arguments **-l** and **/home**.
- The parse-tree program represents the command as the root and the remaining tokens as arguments.
- The submitted parse-tree execution reports successful syntax validation and generation of the execution structure.

Result :

The SKILL-4 programs successfully demonstrate command tokenization and a simple command parse-tree representation. The submitted outputs show correct token separation for **history** and **ls -l /home**, followed by successful syntax validation and execution-structure generation for the parse-tree program.

SKILL-4 Completed Successfully